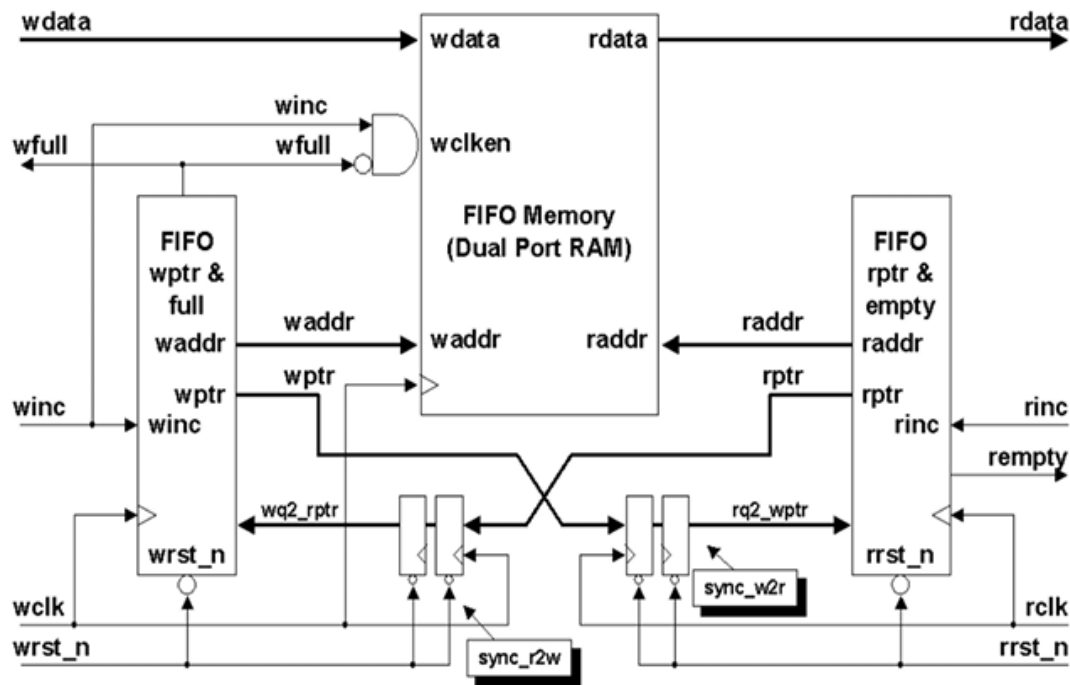


Asynchronous FIFO Design and Python Verification



Name: Mohamed Ahmed Mohamed Hussein

Asynchronous FIFO – Overview and Importance

An **Asynchronous FIFO (First-In First-Out buffer)** is a dual-clock memory structure designed to safely transfer data between two subsystems operating under **different and unrelated clock domains**. Unlike a synchronous FIFO—which assumes a single global clock—an asynchronous FIFO uses **separate write and read clocks**, each potentially running at completely different frequencies or phases.

The core challenge of asynchronous design is maintaining data integrity when clock signals do not align. Asynchronous FIFOs solve this problem by using:

- *Dual-port RAM for simultaneous read/write*
- *Gray-coded read and write pointers*
- *Two-stage (or more) synchronizers for safe pointer transfer across domains*
- *Robust control logic to generate FULL and EMPTY flags*

Importance of Asynchronous FIFO in Digital Systems

Crucial for CDC (Clock Domain Crossing)

Clock Domain Crossing is one of the most error-prone areas in digital design. Metastability, data loss, and incoherent sampling can occur if proper synchronization structures are not used.

Asynchronous FIFOs play a **central role** in CDC because they:

- Allow seamless transfer of multi-bit data across domains
- Prevent metastability by using synchronized Gray pointers
- Decouple timing between producer and consumer modules
- Support any frequency ratio (fast→slow or slow→fast)

In essence, **the asynchronous FIFO is the safest and most scalable CDC mechanism** for streaming data.

Applications of Asynchronous FIFOs

Asynchronous FIFOs appear in almost every modern digital system that involves multiple clock sources, such as:

✓ System-on-Chip (SoC) designs

Different IP blocks (UART, SPI, Ethernet, Video, CPU) often run at independent clocks.

✓ Communication interfaces

Bridging different protocols or clock domains, e.g.

- UART $\longleftrightarrow$ Processor
- Ethernet MAC $\longleftrightarrow$ Internal Bus
- USB $\longleftrightarrow$ DMA controller

✓ High-speed data streaming

Used in image processing pipelines, audio/video FIFOs, DSP engines.

✓ FPGA and ASIC integration

Needed whenever two IP blocks operate with unrelated clocks.

✓ Multi-core and heterogeneous systems

CPU, GPU, DSP, and accelerator blocks often communicate through asynchronous channels.

Project Purpose and Verification Objective

The purpose of this project is to **verify an Asynchronous FIFO design** using two complementary verification approaches:

1. Traditional RTL Verification using Verilog (QuestaSim)

As the baseline method, the FIFO is first verified using a **standard Verilog testbench**. This includes directed and random tests implemented inside QuestaSim. It demonstrates the conventional approach used in industry for functional verification.

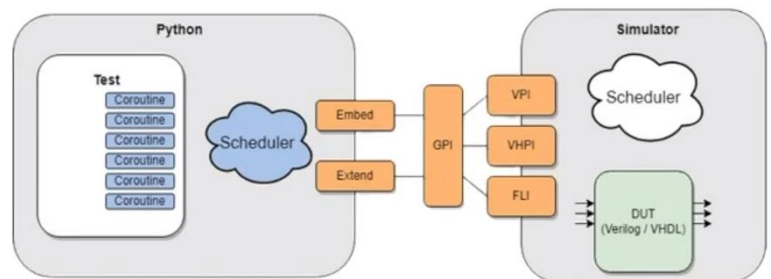
2. Full Python-Based Verification Environment using Cocotb

A modern verification framework was created using **cocotb**, where:

- The entire test environment is written in Python
- Random stimulus generation, monitors, scoreboarding, and functional coverage are implemented in Python
- Cocotb drives the Verilog DUT directly through VPI

This method allows:

- ✓ Higher productivity
- ✓ Reuse of Python libraries
- ✓ Faster development cycles
- ✓ More flexible randomized testing
- ✓ Coverage collection using cocotb-coverage



3. Simulation Flow and Tools

The project demonstrates two verification flows:

- ✓ QuestaSim (Verilog TB)

Used to execute the traditional RTL testbench.

✓ Icarus Verilog + Cocotb (Python TB)

Used to run the full Python verification environment.

Results can then be viewed using:

- **GTKWave** for signal visualization
- **Console-based coverage and scoreboard results** for Python TB

This comparison showcases the **strength of cocotb** as a modern, flexible verification framework while maintaining compatibility with open-source simulators like **Icarus Verilog**.

Project Impact

By verifying the asynchronous FIFO using both methods, the project:

- Demonstrates how CDC-related issues can be caught using robust verification
- Shows that Python-based verification is a strong alternative to traditional testbenches
- Highlights the importance of functional coverage and self-checking environments
- Provides a scalable verification environment applicable to many IP cores involving CDC

This makes the project an excellent demonstration of both **digital design understanding** (asynchronous FIFO and CDC) and **modern verification methodology** (cocotb).

Asynchronous FIFO Specifications

General Asynchronous FIFO Specs

- Designed to transfer data **between two different clock domains** safely.
- Uses **dual-clock architecture**:
 - Independent Write Clock Domain
 - Independent Read Clock Domain
- Employs **Gray-coded read/write pointers** to minimize metastability during pointer synchronization.
- FULL and EMPTY flags are generated using **synchronized pointers**, ensuring safe status detection across domains.

- Supports continuous streaming where write and read operations can occur **simultaneously**.
- Provides **Clock Domain Crossing (CDC) reliability**, preventing data corruption due to metastability.
- Commonly used in:
 - SoC interconnects
 - High-speed serial interfaces
 - Network buffering
 - DMA engines
 - Processor-peripheral communication

Project-Specific FIFO Specs

Clock Specifications

- **Write Clock (W_CLK):**
 - Period = 10 ns
 - Frequency = **100 MHz**
- **Read Clock (R_CLK):**
 - Period = 25 ns
 - Frequency = **40 MHz**
- The write domain is **2.5× faster** than the read domain.

Data Transfer Requirement

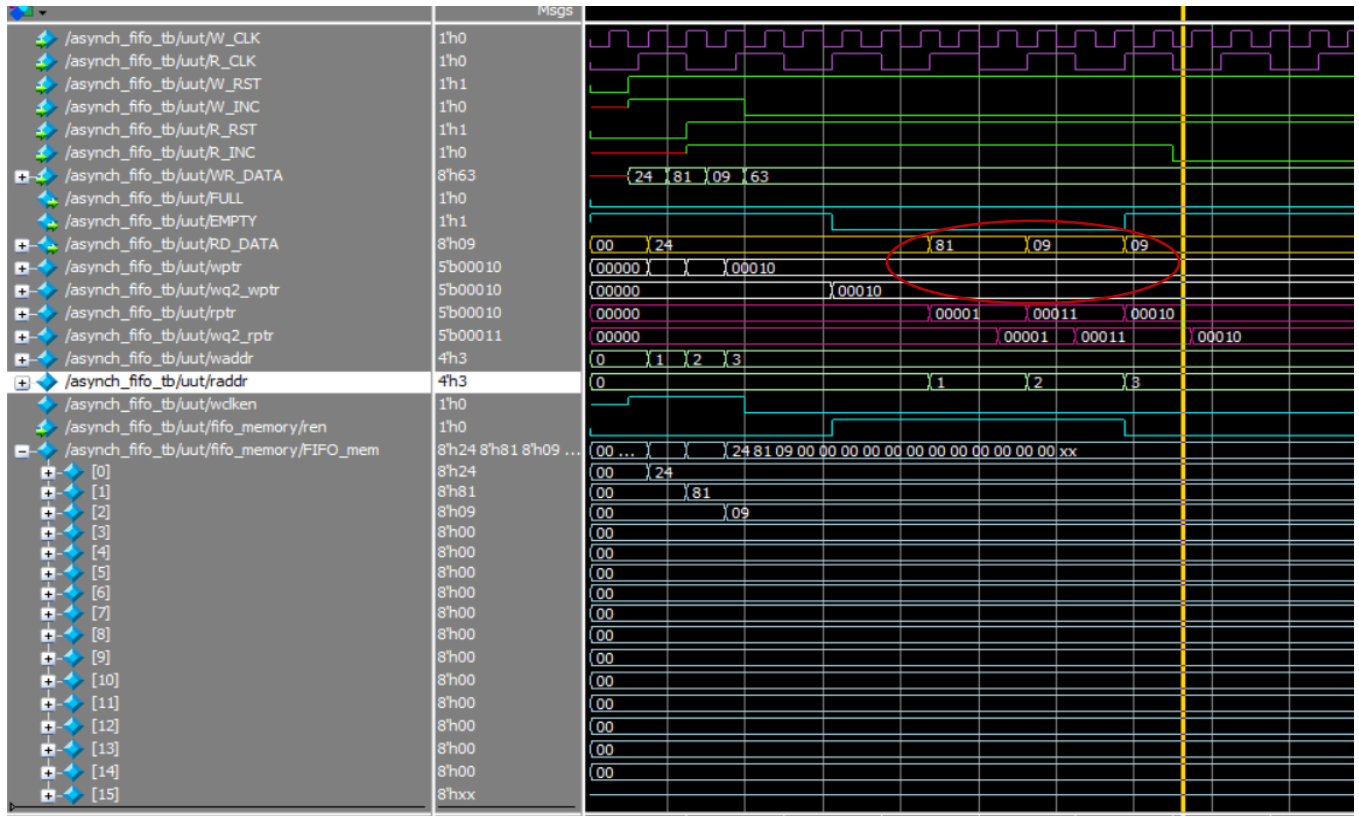
- **The FIFO must reliably transfer 9 bytes from the faster 100 MHz domain to the slower 40 MHz domain.**
- **Because the write side generates data much faster, the FIFO must be deep enough to absorb bursts without overflowing.**

Depth Selection Notes

- Minimum calculated depth $\approx$ **6 entries** (theoretical requirement).
- Depth **8 caused overflow** during simultaneous read+write operations due to:
 - write-rate dominance
 - pointer synchronization latency
 - CDC timing uncertainty
- Final FIFO Depth = **16 entries**, chosen to:
 - ensure safe margin
 - eliminate overflow
 - handle worst-case CDC behavior
 - provide stable FULL/EMPTY flag operation

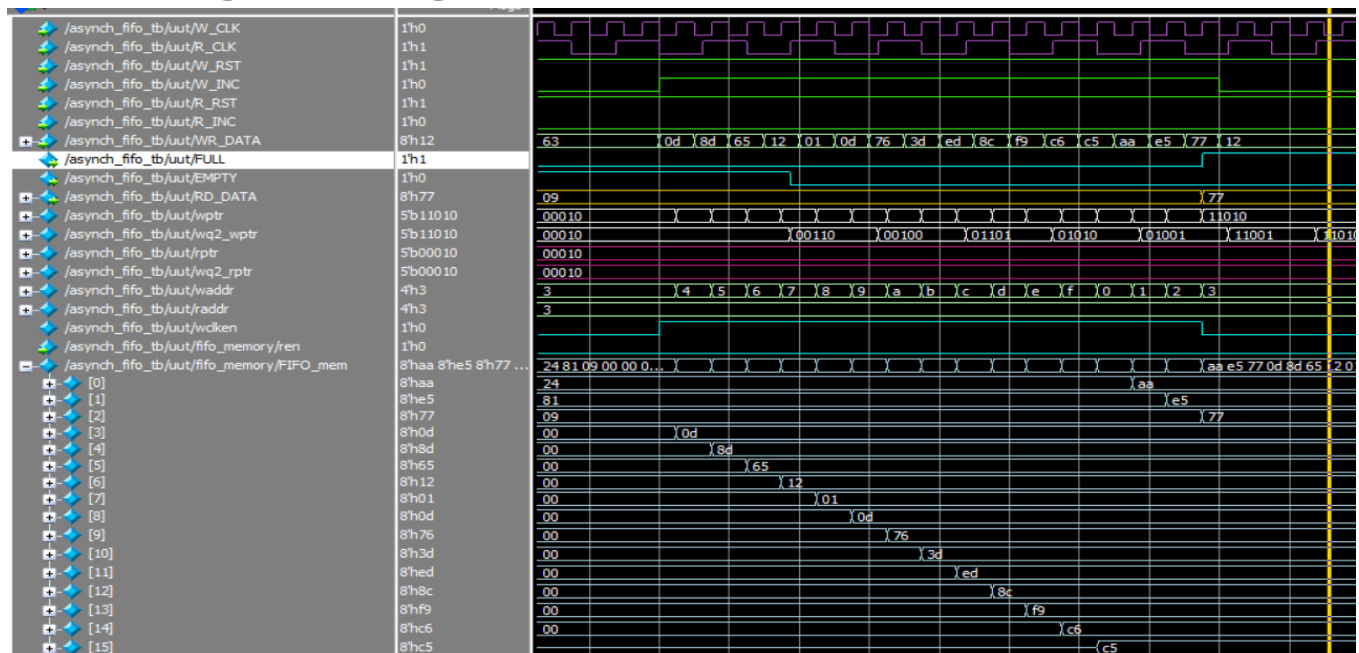
Verilog Testbench (QuestaSim)

- Write 3 read 3 (regular case):



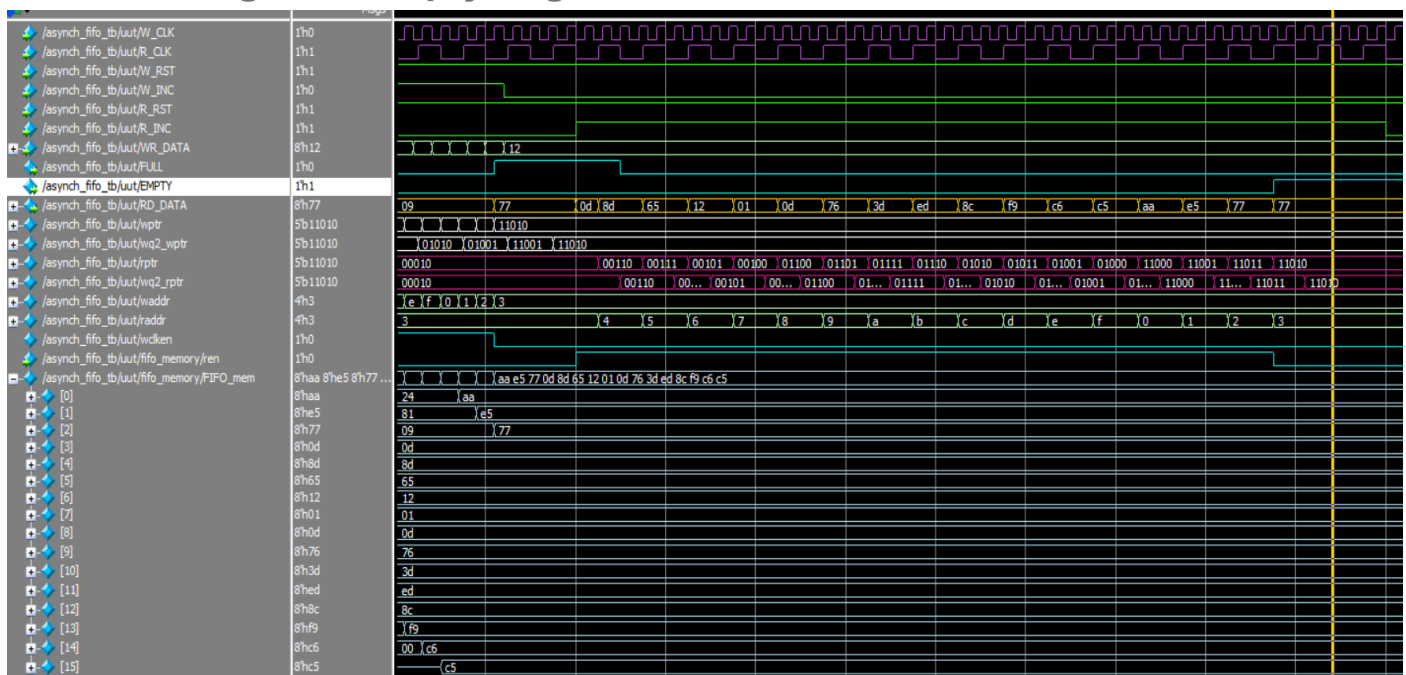
Empty flag went down till we read the 3 values and then went up again

- Writing till full flag rises:



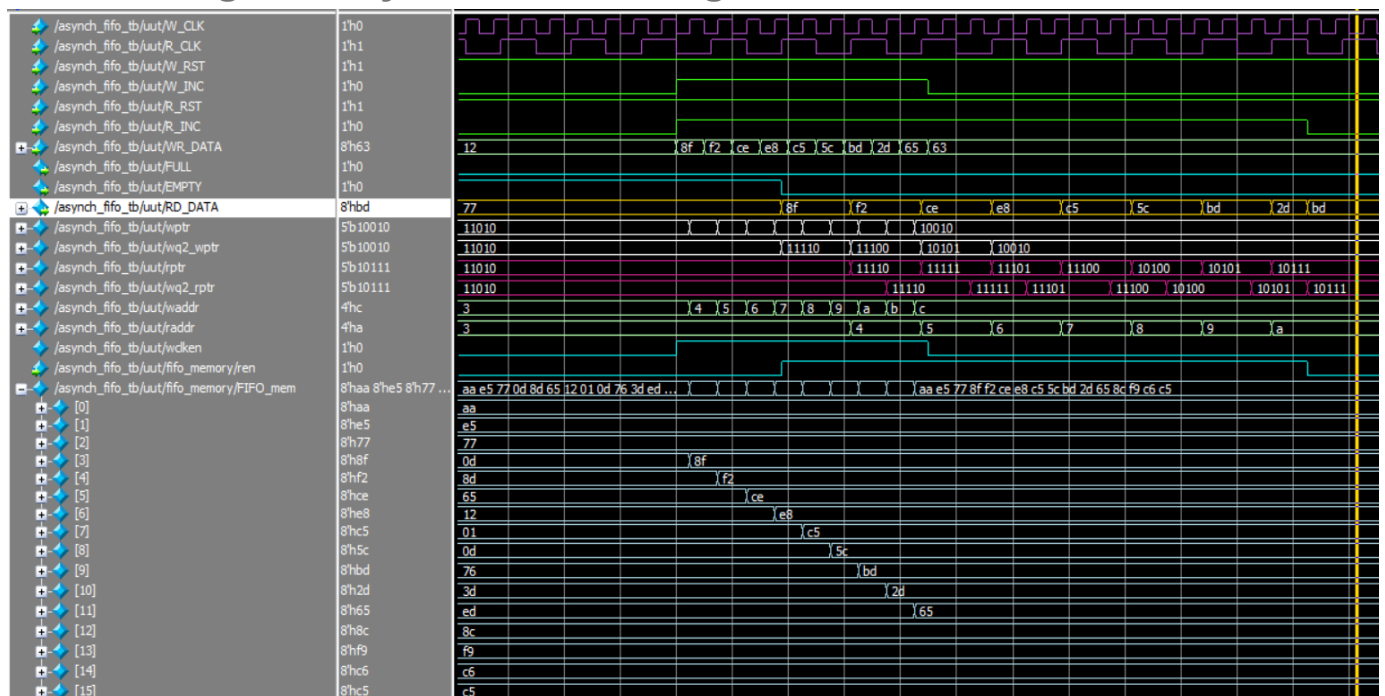
when full flag rises, we don't **overwrite** any value that hasn't been read yet

- Reading until empty flag rises:



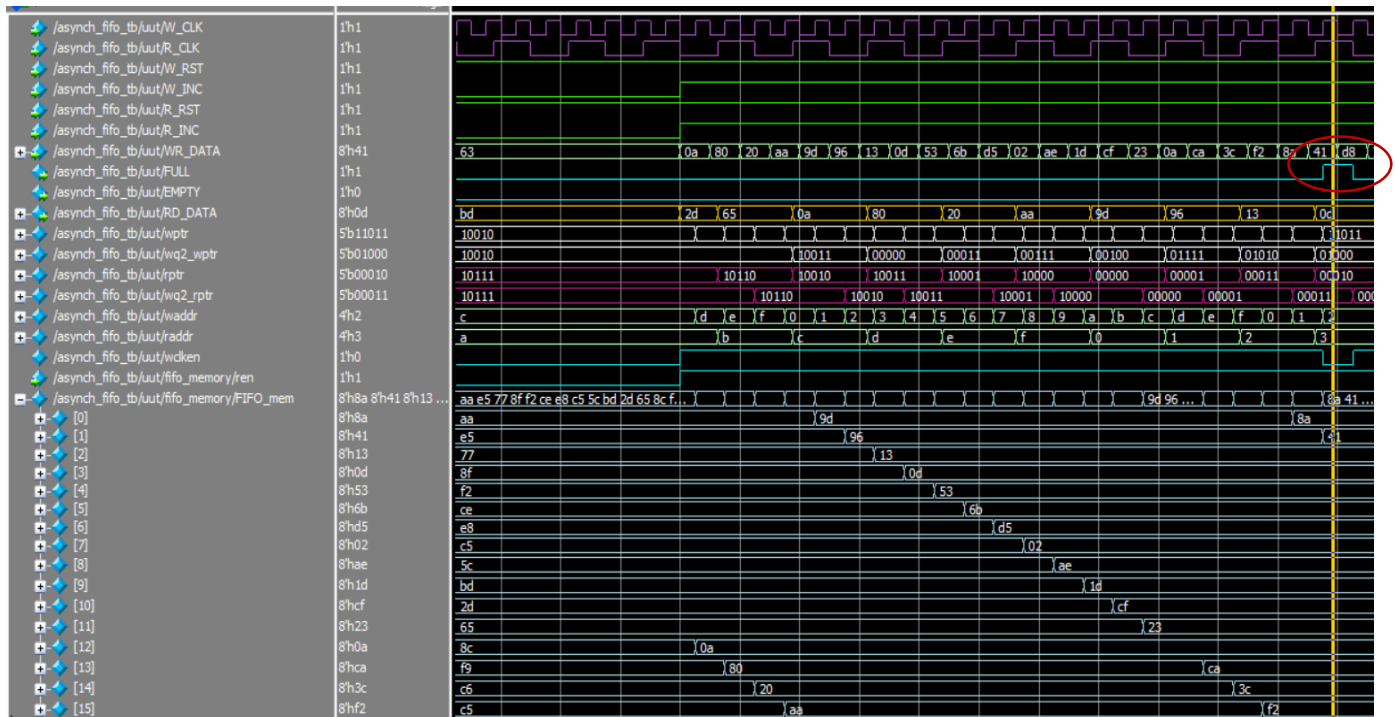
We read all values and empty flag raised

- Writing the 9 bytes while reading at the same time:



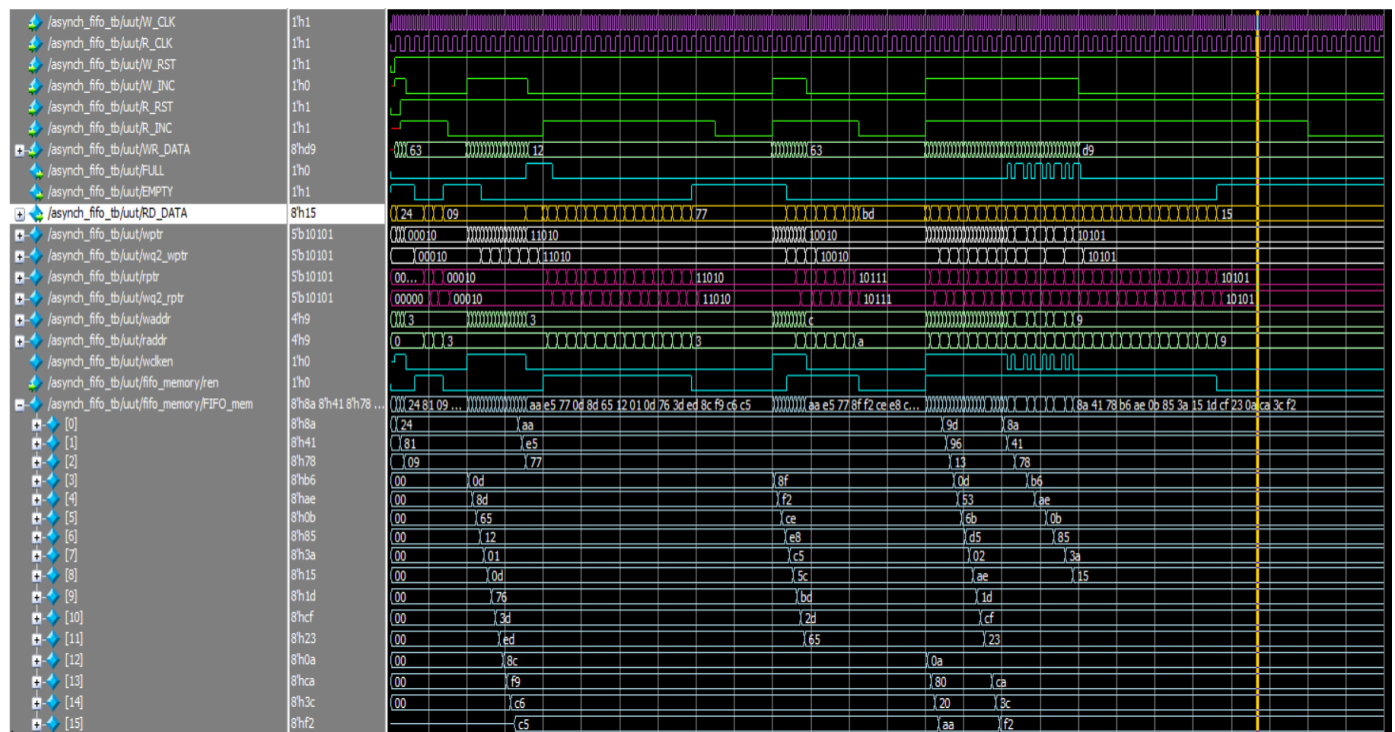
we read the same values written and the **full flag didn't rise** which indicates the **correctness of our depth calculation**

- Writing values till convergence (full flag should rise and fall):



Full flag raised at the 22th value written so we have a **margin of 13 values for safety** (we can send 22 bytes without violation)

- Full waveform:



Note: in my test I **used multiple initial blocks** each pair is for a **read-write tests**, the initial blocks act as a **fork-join in SV**

```
// write stimulus
initial begin
    // task for write reset
    write_reset();
    infering_data(1); // (write enable) only write enabled, random data
    infering_data(1);
    infering_data(1);
    infering_data(0);
    #150; // waiting until reading is done
    // testing full
    for(j = 0; j < FIFO_DEPTH ; j = j + 1) begin
        infering_data(1);
    end
    infering_data(0);
end

// read stimulus
initial begin
    // task for read reset
    read_reset();
    reading_data(1); // (read enable) only read enabled
    check_data(); // always check data after reading
    reading_data(1);
    check_data();
    reading_data(1);
    check_data();
    #50;
    reading_data(0);
    #250;
    // testing empty
    for(k = 0; k < FIFO_DEPTH ; k = k + 1) begin
        reading_data(1);
        check_data();
    end
    #50;
    reading_data(0);
    #100;
end
```

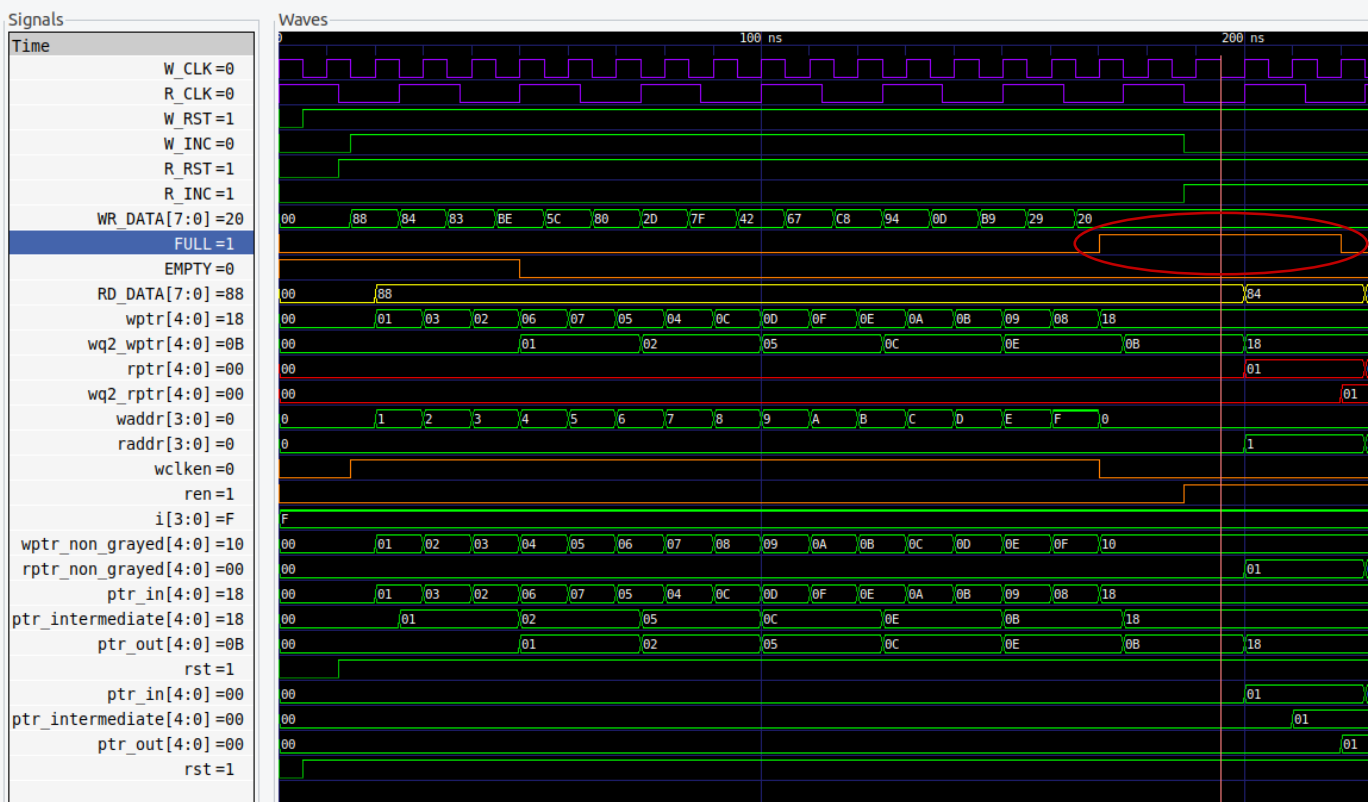
For more check [ASYNCHRONOUS_FIFO_TB.v](#)

Transcript:

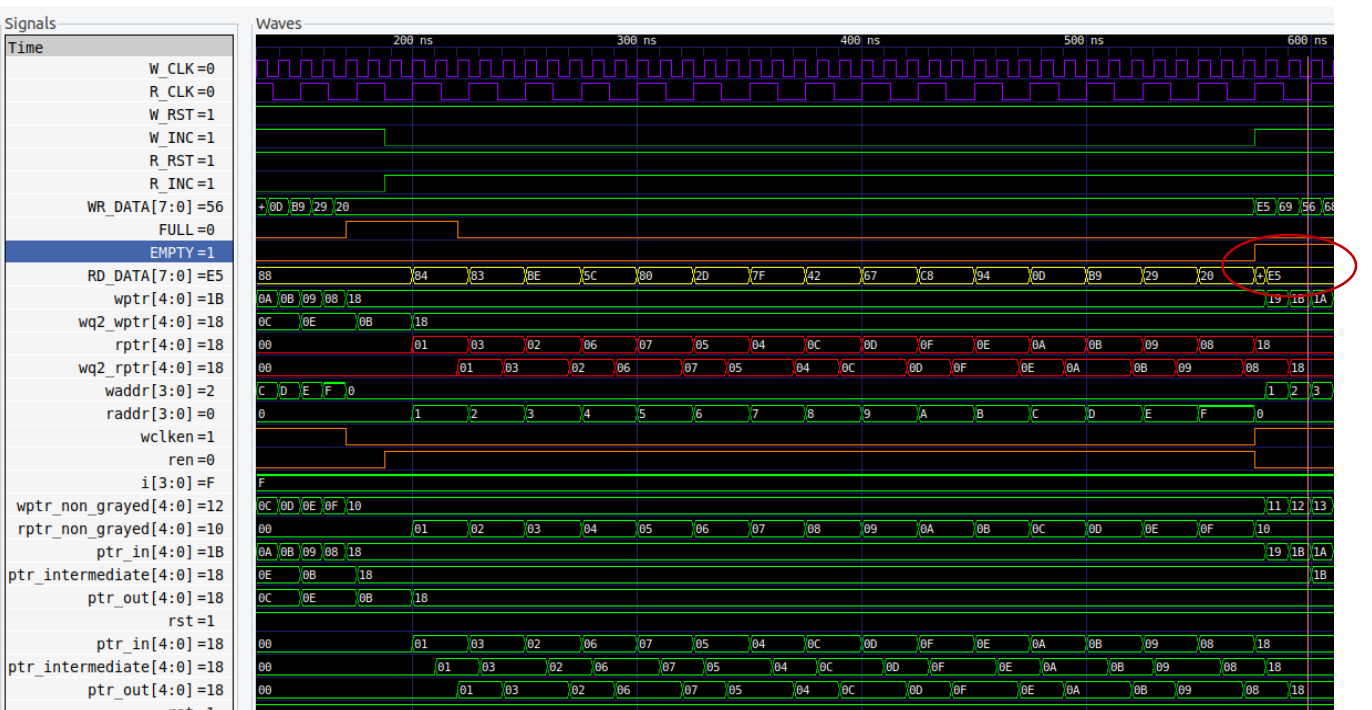
```
Data match! WR_DATA was : 23, RD_DATA: 23
Data match! WR_DATA was : 0a, RD_DATA: 0a
Data match! WR_DATA was : ca, RD_DATA: ca
Data match! WR_DATA was : 3c, RD_DATA: 3c
Data match! WR_DATA was : f2, RD_DATA: f2
Data match! WR_DATA was : 8a, RD_DATA: 8a
Data match! WR_DATA was : 41, RD_DATA: 41
Data match! WR_DATA was : 78, RD_DATA: 78
Data match! WR_DATA was : b6, RD_DATA: b6
Data match! WR_DATA was : ae, RD_DATA: ae
Data match! WR_DATA was : 0b, RD_DATA: 0b
Data match! WR_DATA was : 85, RD_DATA: 85
Data match! WR_DATA was : 3a, RD_DATA: 3a
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
Data match! WR_DATA was : 15, RD_DATA: 15
```

Class-based python verification (Cocotb + Icarus)

- Writing till full flag rises:

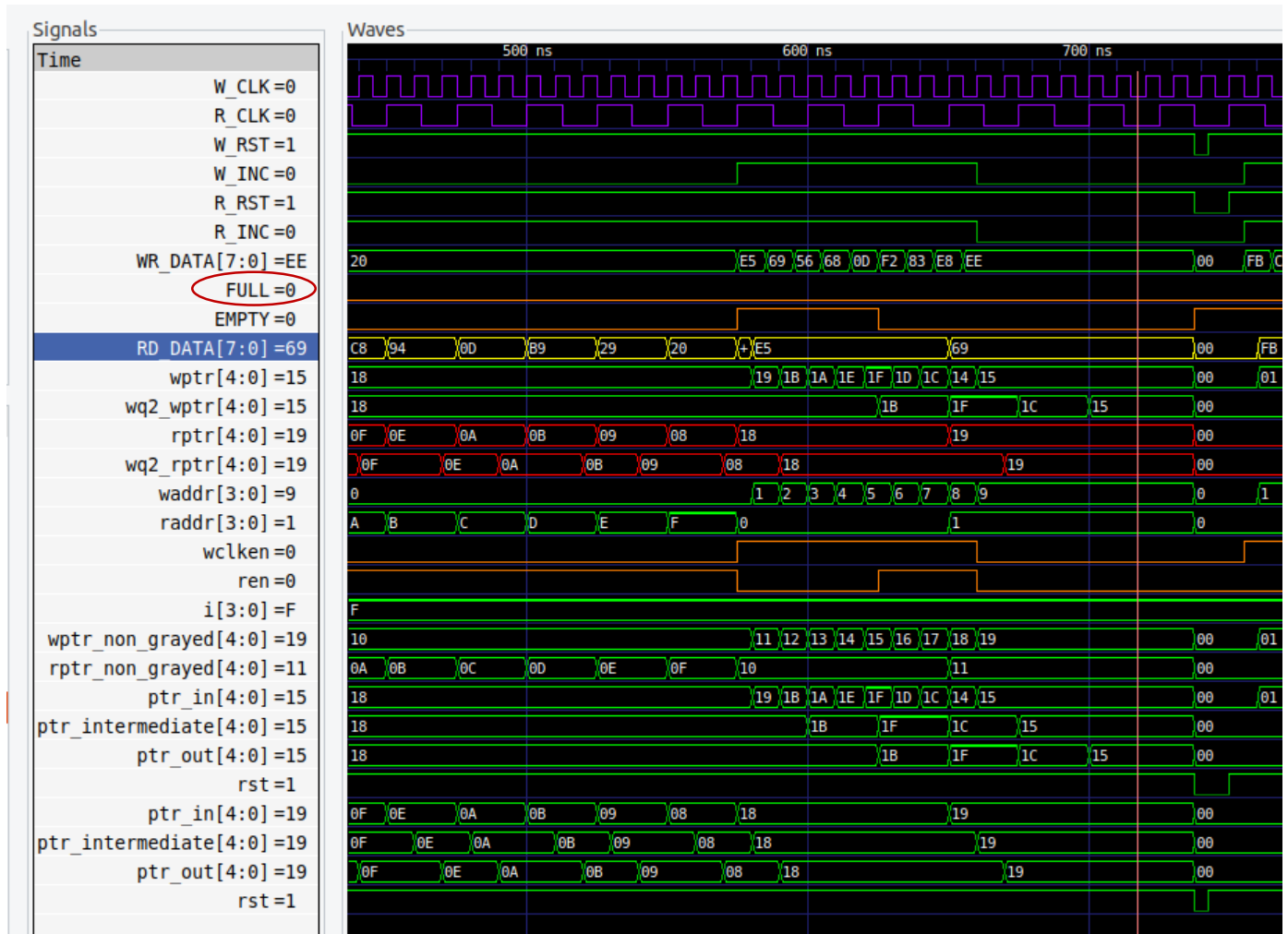


- Reading until empty flag rises:



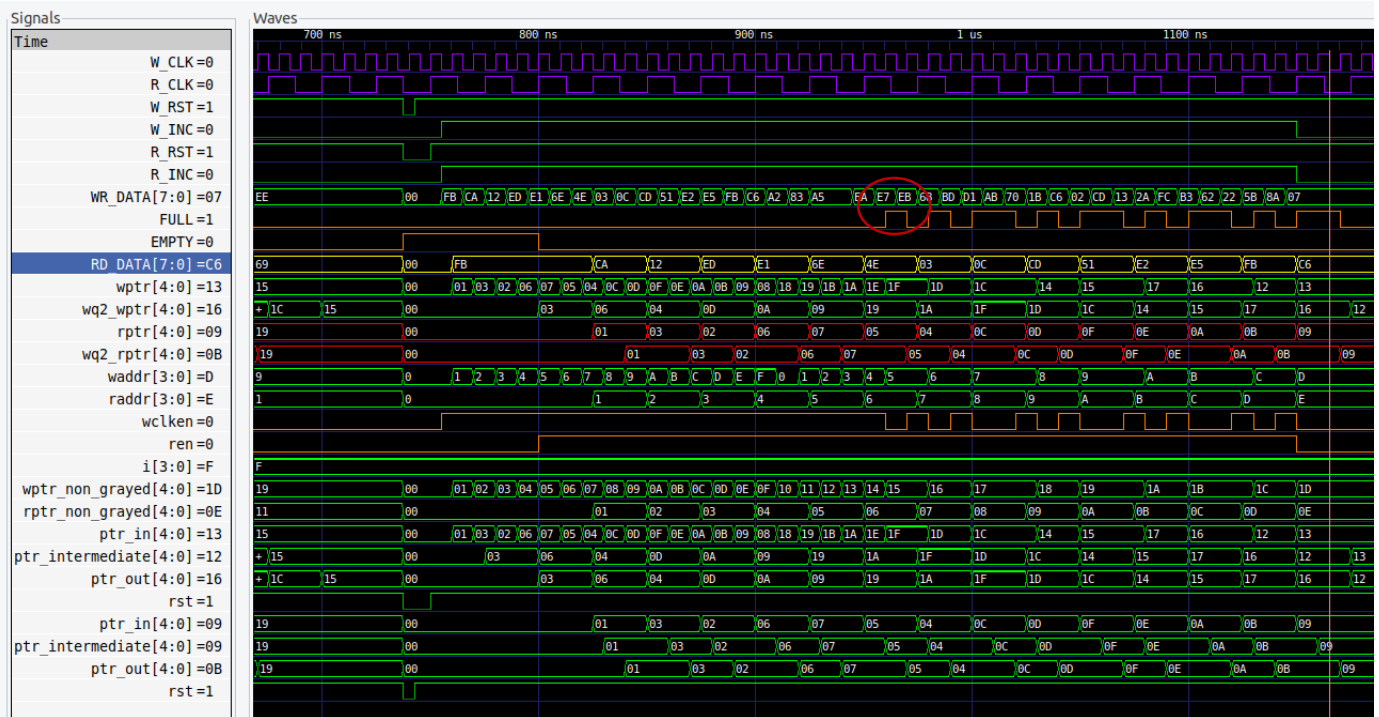
Notice how the **Write_Data** from the last snapshot is **now displayed** on the **Read_Data** signal till the *empty* is high

- Writing the 9 bytes while reading at the same time:

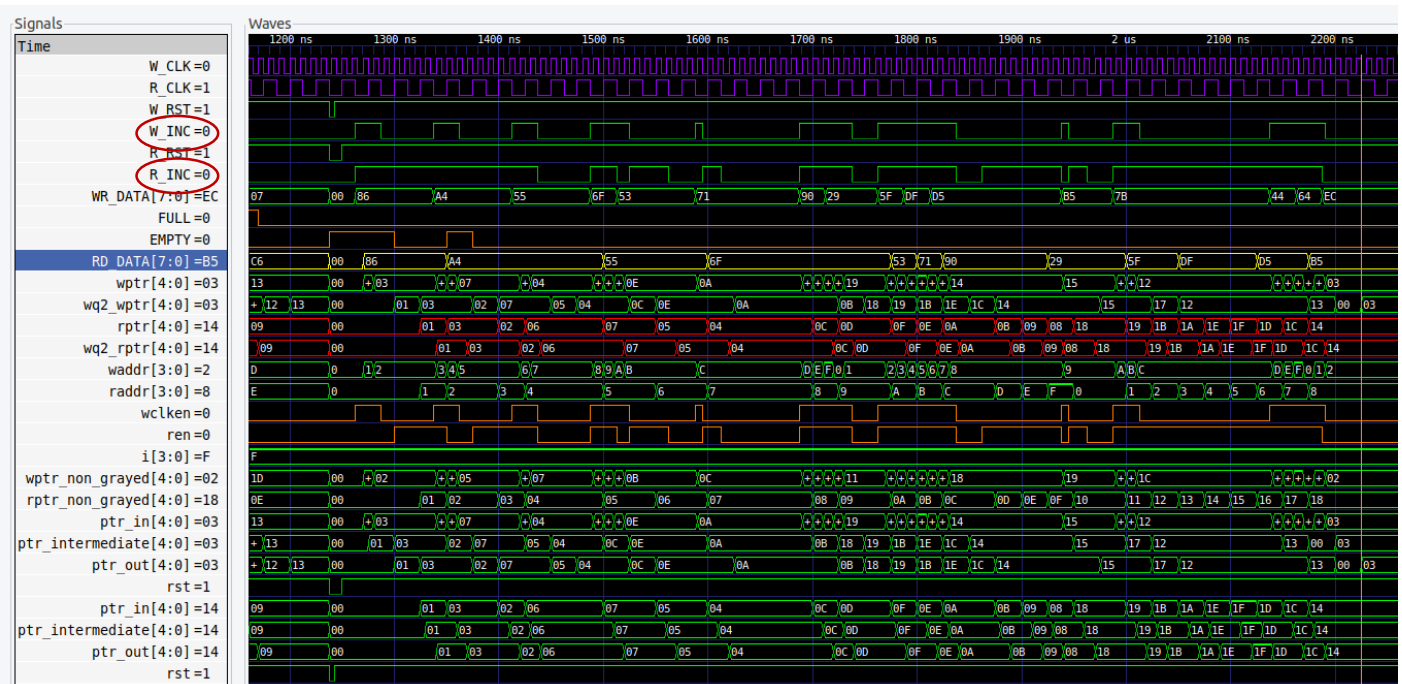


we read the same values written and the **full flag didn't rise** which indicates the **correctness of our depth calculation**

- Writing values till convergence (full flag should rise and fall):

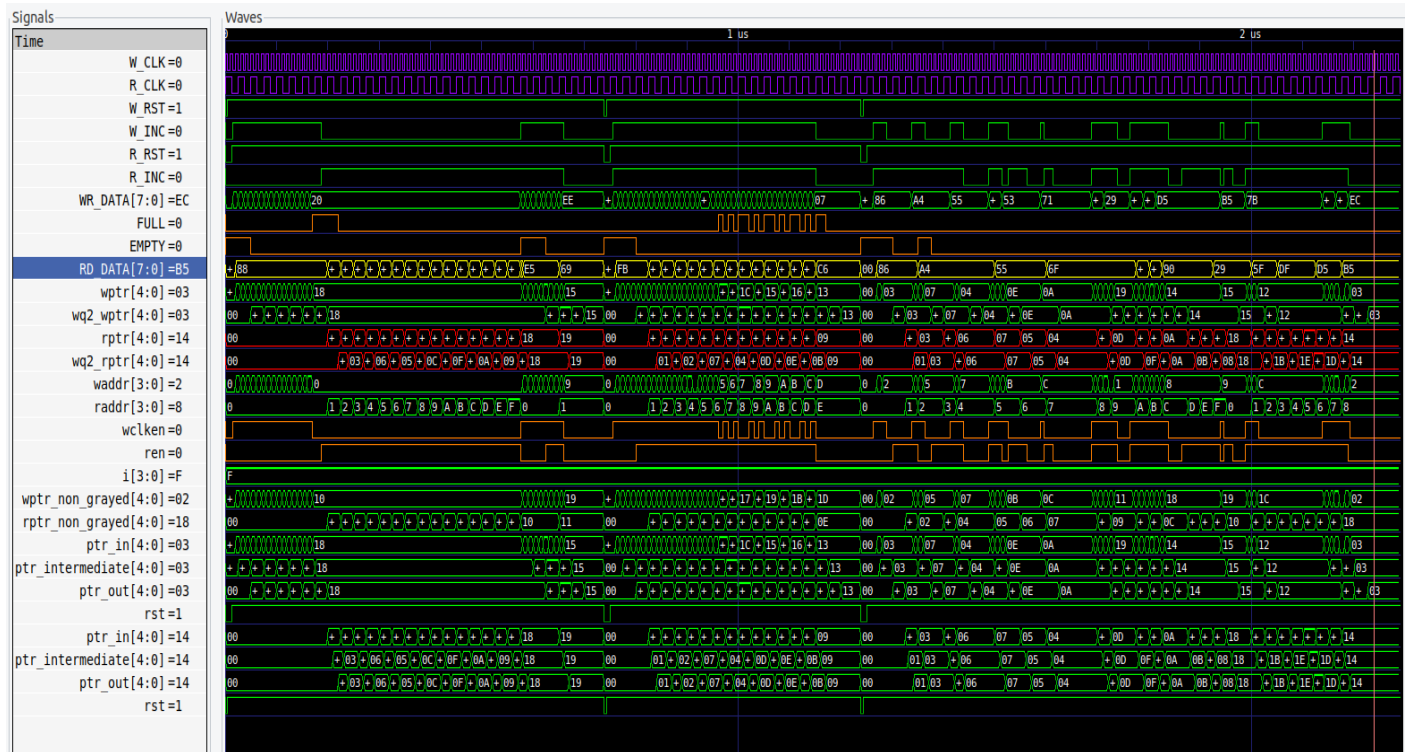


- Full flag raised at the 22th value written so we have a **margin of 13 values for safety** (we can send 22 bytes without violation)
- Reset the FIFO first for better visualization of the *RD_DATA* with *WR_DATA*
- Random Stimulus with more reading:



Notice how the R_INC goes high more often than W_INC as I made it **70:30** in the [Transaction.py](#), also that is obvious that the full flag didn't rise

- Full waveform:



Transaction:

```
import random

class FifoTransaction:
    def __init__(self, dut, data_width=8):
        self.dut = dut
        self.data_width = data_width
        self.wr_en = 0
        self.rd_en = 0
        self.wr_data = 0
        self.full = 0
        self.empty = 0
        self.rd_data = 0

    async def randomize(self):
        # Randomize write/read enables
        # Weighted Randomization for Read Enable (Read more often than not)
        # Weights: [0, 1] -> [30, 70] means 70% chance of read_en=1
        self.rd_en = random.choices([0, 1], weights=[30, 70], k=1)[0]
```

```

# Weighted Randomization for Write Enable (Write less often than read)
# Weights: [0, 1] -> [70, 30] means 30% chance of wr_en=1
self.wr_en = random.choices([0, 1], weights=[70, 30], k=1)[0]
# Randomize input data only if write enabled
if self.wr_en:
    self.wr_data = random.randint(0, (1 << self.data_width) - 1)
return self

async def direct(self, w_flag, r_flag):
    self.wr_en = w_flag
    self.rd_en = r_flag
    # Randomize input data only if write enabled
    if self.wr_en:
        self.wr_data = random.randint(0, (1 << self.data_width) - 1)
    return self

def __repr__(self):
    return f"Txn(wr_en={self.wr_en}, rd_en={self.rd_en},
wr_data={self.wr_data})"

```

- I meant to make the reading process happens more than writing as the reading domain is the **slower one!**
- You may check [transaction.py](#)

Coverage

```

==== FIFO Coverage Report ====
2287.50ns INFO cocotb.test fifo : <cocotb_coverage.coverage.CoverItem object at 0x77b552344e20>, coverage=8, size=8
2287.50ns INFO cocotb.test fifo.empty_flag : <cocotb_coverage.coverage.CoverPoint object at 0x77b552346ce0>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN not_empty : 80
2287.50ns INFO cocotb.test BIN empty : 11
2287.50ns INFO cocotb.test fifo.full_flag : <cocotb_coverage.coverage.CoverPoint object at 0x77b552346d10>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN not_full : 89
2287.50ns INFO cocotb.test BIN full : 2
2287.50ns INFO cocotb.test fifo.read_trans : <cocotb_coverage.coverage.CoverPoint object at 0x77b552346d40>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN read_disabled : 29
2287.50ns INFO cocotb.test BIN read_enabled : 62
2287.50ns INFO cocotb.test fifo.write_trans : <cocotb_coverage.coverage.CoverPoint object at 0x77b552344df0>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN write_disabled : 52
2287.50ns INFO cocotb.test BIN write_enabled : 39

```

- You may check [coverage_fifo.xml](#) for clear text
- **Code:**

```

import cocotb
from cocotb_coverage.coverage import *

@coverage_section
class FifoCoverage:
    def __init__(self, dut):
        self.dut = dut

```

```

@CoverPoint(
    "fifo.write_trans",
    xf=lambda txn: txn.wr_en,
    bins=[0, 1],
    bins_labels=["write_disabled", "write_enabled"]
)
@CoverPoint(
    "fifo.read_trans",
    xf=lambda txn: txn.rd_en,
    bins=[0, 1],
    bins_labels=["read_disabled", "read_enabled"]
)
@CoverPoint(
    "fifo.full_flag",
    xf=lambda txn: txn.full,
    bins=[0, 1],
    bins_labels=["not_full", "full"]
)
@CoverPoint(
    "fifo.empty_flag",
    xf=lambda txn: txn.empty,
    bins=[0, 1],
    bins_labels=["not_empty", "empty"]
)
def sample_coverage(self, txn):
    """Sample coverage using the given transaction object."""
    pass # decorators handle sampling automatically

def report(self, filename="coverage_report.txt"):
    """Generate a coverage report (console + file)."""
    log = cocotb.logging.getLogger("cocotb.test")
    print("\n==== FIFO Coverage Report ====")
    coverage_db.report_coverage(log.info, bins=True)
    coverage_db.export_to_xml(filename="coverage_fifo.xml")
    print(f"[INFO] Coverage report saved to {filename}")

```

Transcript

```
make[1]: Leaving directory '/home/icpedia/Coco_tb/Asynchronous_FIFO_Cocotb'
(cocotb_env) icpedia@icpedia-virtual-machine:~/Coco_tb/Asynchronous_FIFO_Cocotb$ make
rm -f results.xml
"make" -f Makefile results.xml
make[1]: Entering directory '/home/icpedia/Coco_tb/Asynchronous_FIFO_Cocotb'
rm -f results.xml
MODULE=asynchronous_fifo_test TESTCASE= TOPLEVEL=asynch_fifo TOPLEVEL_LANG=verilog \
  /usr/bin/vvp -M /home/icpedia/cocotb_env/lib/python3.10/site-packages/cocotb/libs -m libcocotbvpi_icarus sim_build/sim.vvp
...ns INFO gpi ..mbed/gpi_embed.cpp:108 in set_program_name_in_venv Using Python virtual environment interpreter
cocotb_env/bin/python
...ns INFO gpi ../gpi/CpiCommon.cpp:101 in gpi_print_registered_impl VPI registered
0.00ns INFO cocotb Running on Icarus Verilog version 11.0 (stable)
0.00ns INFO cocotb Running tests with cocotb v1.9.2 from /home/icpedia/cocotb_env/lib/python3.10/site-packages/cocotb
0.00ns INFO cocotb Seeding Python random module with 1763144149
0.00ns INFO cocotb.regression pytest not found, install it to enable better AssertionError messages
0.00ns INFO cocotb.regression Found test asynchronous_fifo_test.async_fifo_test
0.00ns INFO cocotb.regression running_async_fifo_test (1/1)
                                Test asynchronous FIFO with directed + random transactions
VCD info: dumpfile dump.vcd opened for output.
/home/icpedia/Coco_tb/Asynchronous_FIFO_Cocotb/asynchronous_fifo_test.py:55: RuntimeWarning: coroutine 'FifoScoreboard.check_data' was never awaited
scoreboard.check_data()
RuntimeWarning: Enable tracemalloc to get the object allocation traceback

==== FIFO Coverage Report ====
2287.50ns INFO cocotb.test fifo : <cocotb_coverage.coverage.CoverItem object at 0x77b552344e20>, coverage=8, size=8
2287.50ns INFO cocotb.test fifo.empty_flag : <cocotb_coverage.coverage.CoverPoint object at 0x77b552346ce0>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN not_empty : 80
2287.50ns INFO cocotb.test BIN empty : 11
2287.50ns INFO cocotb.test fifo.full_flag : <cocotb_coverage.coverage.CoverPoint object at 0x77b552346d10>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN not_full : 89
2287.50ns INFO cocotb.test BIN full : 2
2287.50ns INFO cocotb.test fifo.read_trans : <cocotb_coverage.coverage.CoverPoint object at 0x77b552346d40>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN read_disabled : 29
2287.50ns INFO cocotb.test BIN read_enabled : 62
2287.50ns INFO cocotb.test fifo.write_trans : <cocotb_coverage.coverage.CoverPoint object at 0x77b552344df0>, coverage=2, size=2
2287.50ns INFO cocotb.test BIN write_disabled : 52
2287.50ns INFO cocotb.test BIN write_enabled : 39
[INFO] Coverage report saved to coverage_report.txt
2287.50ns INFO cocotb Asynchronous FIFO test PASSED
2287.50ns INFO cocotb.async_fifo_test passed
*****
** TEST STATUS SIM TIME (ns) REAL TIME (s) RATIO (ns/s) **
*****
** asynchronous_fifo_test.async_fifo_test PASS 2287.50 0.08 29259.39 **
*****
** TESTS=1 PASS=1 FAIL=0 SKIP=0 2287.50 0.09 25681.42 **
*****
```

Our test is **Passed** without errors!